

Claro, pero para mirar el conjunto de datos no necesito R. Los podría mirar desde cualquier otro software. ¿Por qué no calculo estadísticos descriptivos?¹³:

```
summary(datosPoblacion)
sexo      altura      peso      edad
F:6   Min.    :1.675   Min.    :45.72   Min.    :19.74
M:4   1st Qu.:1.689   1st Qu.:72.08   1st Qu.:38.37
      Median :1.748   Median :78.74   Median :40.94
      Mean   :1.760   Mean   :77.91   Mean   :43.72
      3rd Qu.:1.791   3rd Qu.:88.88   3rd Qu.:51.94
      Max.   :1.898   Max.   :95.62   Max.   :69.05
```

Muy lindo, pero difícil para trabajar. Si quiero solo la media del peso:

```
mean(datosPoblacion$peso)
[1] 77.91363
```

¿Y si quiero un vector con la media de todos los campos?:¹⁴

```
sapply(datosPoblacion, mean)
      sexo altura      peso      edad
      NA  1.75955 77.91363 43.72370
Mensajes de aviso perdidos
In mean.default(X[[1L]], ...) :
  argument is not numeric or logical: returning NA
```

¿Y el su desvío estándar?:¹⁵

```
sapply(datosPoblacion, sd)
      sexo altura      peso      edad
      NA  0.08222817 14.92228094 16.56140722
Mensajes de aviso perdidos
In var(as.vector(x), na.rm = na.rm) : NAs introducidos por coerción
```

Quizás me interesaría saber cuánto suman las columnas:

```
colSums(datosPoblacion[, 2:4])
      altura      peso      edad
17.5955 779.1363 437.2370
```

¹³ Para profundizar en el uso de estadísticos descriptivos en R, y acerca de R en general, un buen libro de consultas es “Introducción a R”.

¹⁴ Fijense que, por más que hay un campo que no es numérico, por lo cual no se le puede calcular la media, R calcula la media para todos los otros campos e informa que campo no se pudo calcular.

¹⁵ Resumiendo, *sapply()* nos permite aplicar funciones a cada campo (columna) de un data frame.

O calcular la frecuencia absoluta para alguna variable:

```
table(datosPoblacion$sexo)
```

```
F M  
6 4
```

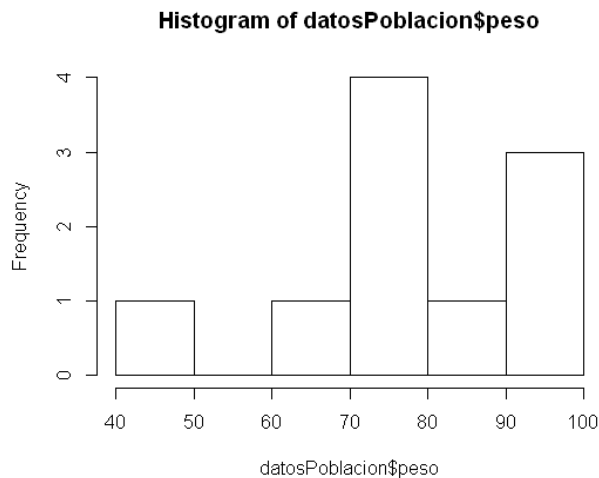
Ahora bien, saber la media de la altura es interesante, ¿pero será la misma por cada sexo?:¹⁶

```
tapply(datosPoblacion$altura, datosPoblacion$sexo, mean)
```

```
F M  
1.753358 1.768837
```

Ahora, muy lindo todo, ¿por qué no hacemos algunos gráficos?:

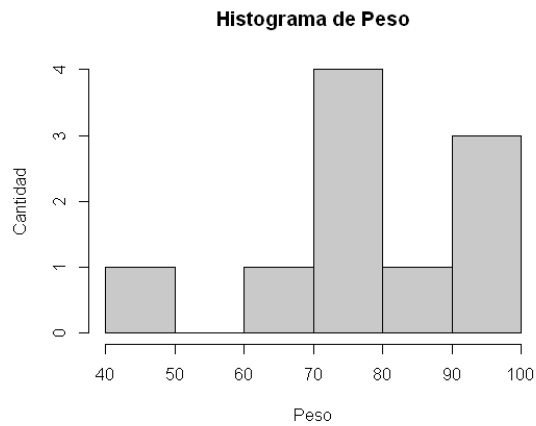
```
hist(datosPoblacion$peso)
```



Muy feo, ¿no?:

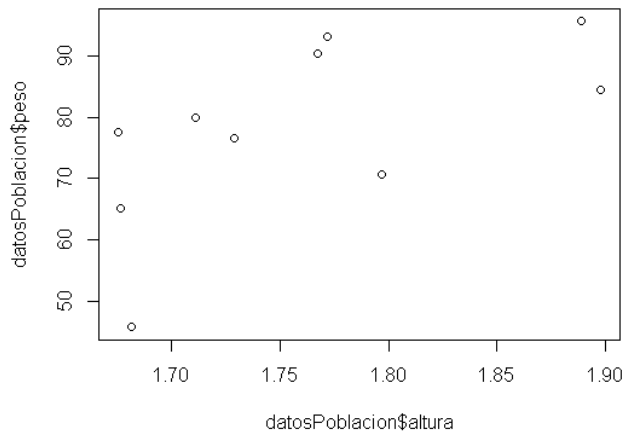
```
hist(datosPoblacion$peso, col = "cyan", main="Histograma de  
Peso", xlab = "Peso", ylab = "Cantidad", freq = TRUE)
```

¹⁶ La función *tapply()* nos permite aplicar una función (el tercer parámetro) a una columna de un data frame (el primer parámetro) discriminando los datos según el valor de otra columna del data frame (el segundo parámetro). En este caso, calculamos la media de la altura discriminando por sexo.



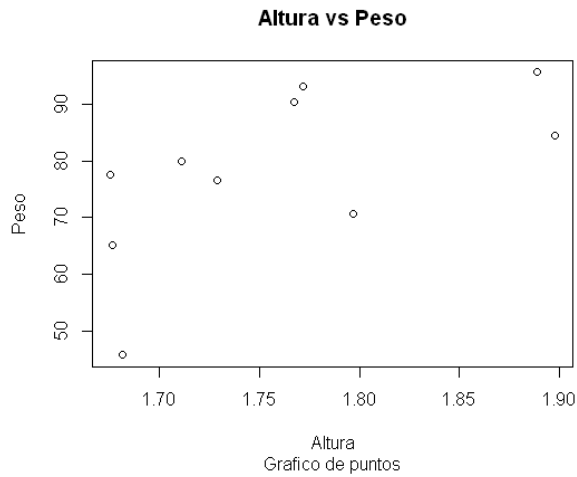
¿Estarán relacionadas la altura y el peso?

```
plot(datosPoblacion$altura, datosPoblacion$peso)
```



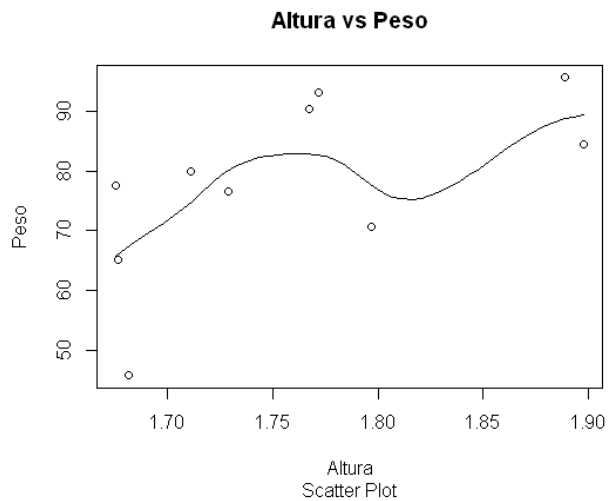
Bueno, podemos mejorarlo:

```
plot(datosPoblacion$altura, datosPoblacion$peso, xlab =
"Altura", ylab = "Peso", main = "Altura vs Peso", type =
"p", sub = "Grafico de puntos")
```



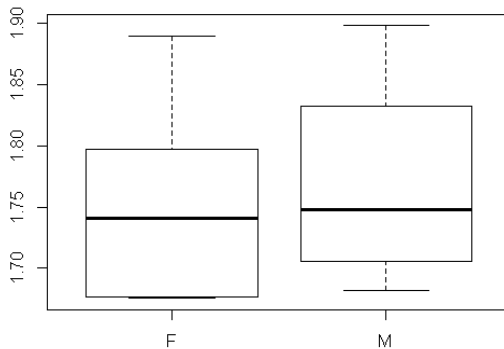
También podemos hacer:

```
scatter.smooth(datosPoblacion$altura, datosPoblacion$peso,
xlab = "Altura", ylab = "Peso", main = "Altura vs Peso",
type = "p", sub = "Scatter Plot")
```



¿Y si graficamos la altura por sexo?:

```
boxplot(datosPoblacion$altura ~ datosPoblacion$sexo)
```

Deberíamos poder guardar nuestro data frame para usarlo en el futuro:

```
write.csv(datosPoblacion, "../data//datosPoblacion.csv")
```

Fíjense que R permite usar referencias relativas a directorios, por lo cual no importa donde tenemos ubicada a nuestra carpeta, sino que mantengamos la estructura adecuada.

Probemos ahora borrar nuestro data frame:

```
rm(datosPoblacion)
datosPoblacion
Error: objeto 'datosPoblacion' no encontrado
```

Como vemos, el objeto “datosPoblacion” no está más en nuestra memoria. Vamos a leerlo y cargarlo en memoria:

```
datosPoblacion <- read.csv("../data//datosPoblacion.csv")
View(datosPoblacion)
```

2.3. Acerca del trabajo con matrices en R

Borremos la memoria de trabajo y carguemos el paquete para trabajar con matrices:

```
rm(list=ls())
library(Matrix)
```

Creemos ahora nuestra primera matriz:¹⁷

```
myMatriz <- matrix(c(1,2,3,4,5,6), nrow = 2, ncol = 3, byrow  
= TRUE)
```

```
myMatriz  
      [,1] [,2] [,3]  
[1,]    1    2    3  
[2,]    4    5    6
```

Podemos nombrar a las filas y columnas:

```
nombreFilas <- c("Fila01", "Fila02")  
nombreColumnas <- c("Col01", "Col02", "Col03")  
dimnames(myMatriz) <- list(nombreFilas, nombreColumnas)  
myMatriz  
      Col01 Col02 Col03  
Fila01    1    2    3  
Fila02    4    5    6
```

Accedo a sus valores usando la notación de "[]":

```
myMatriz[2,3]      # Fila 2, columna 3  
[1] 6  
  
myMatriz[2,]      # Segunda fila  
Col01 Col02 Col03  
    4    5    6  
  
myMatriz[,3]      # Tercera columna  
Fila01 Fila02  
    3    6
```

Si pongo un solo índice:

```
myMatriz[3]  
[1] 2
```

Me devuelve la celda "n", contando primero toda la primera columna, después la segunda, etc. ¹⁸O sea¹⁹:

¹⁷ El primer parámetro de *matrix()*, el vector "c(1,2,3,4,5,6)" contiene a los elementos de la matriz. Los mismos se ordenan de acuerdo al valor del parámetro "byrow". Si vale TRUE, los elementos se cargan completando filas y pasando luego a las columnas. Prueben generar la misma matriz pero con el parámetro "byrow" tomando el valor FALSE, verán que la misma se llena por columnas.

¹⁸ Si, sin importar con qué valor de "byrow" la creé.

```
n <- 4

myMatriz[n] == myMatriz[nrow(myMatriz)-(n %%
nrow(myMatriz)) , (n-1) %% nrow(myMatriz)+1]
```

```
n <- 5

myMatriz[n] == myMatriz[nrow(myMatriz)-(n %%
nrow(myMatriz)) , (n-1) %% nrow(myMatriz)+1]
```

```
n <- 2

myMatriz[n] == myMatriz[nrow(myMatriz)-(n %%
nrow(myMatriz)) , (n-1) %% nrow(myMatriz)+1]
```

```
n <- 6

myMatriz[n] == myMatriz[nrow(myMatriz)-(n %%
nrow(myMatriz)) , (n-1) %% nrow(myMatriz)+1]
```

Como tipo de dato, tiene sus métodos `is.matrix` y `as.matrix`:

```
is.matrix(myMatriz)

myVector <-c(1,2,3)

is.vector(myVector)

is.matrix(myVector)

is.matrix(as.matrix(myVector))
```

`matrix` es un tipo básico disponible sin necesidad de cargar el paquete "Matrix", pero este permite disponer de toda la funcionalidad para el trabajo con matrices:

```
myOtraMatriz <- matrix(10 + seq(1:28), 4, 7, byrow = TRUE)
```

```
myOtraMatriz
  [,1] [,2] [,3] [,4] [,5] [,6] [,7]
[1,]  11  12  13  14  15  16  17
[2,]  18  19  20  21  22  23  24
[3,]  25  26  27  28  29  30  31
[4,]  32  33  34  35  36  37  38
```

¹⁹ El operador "==" sirve, como ya vimos, para realizar comparaciones, devolviendo un valor del tipo "logical".

```
myOtraMatrizXCol <- matrix(10 + seq(1:28), 4, 7, byrow = FALSE)
```

```
myOtraMatrizXCol
      [,1] [,2] [,3] [,4] [,5] [,6] [,7]
[1,]    11    15    19    23    27    31    35
[2,]    12    16    20    24    28    32    36
[3,]    13    17    21    25    29    33    37
[4,]    14    18    22    26    30    34    38
```

```
myOtraMatriz + myOtraMatrizXCol
      [,1] [,2] [,3] [,4] [,5] [,6] [,7]
[1,]    22    27    32    37    42    47    52
[2,]    30    35    40    45    50    55    60
[3,]    38    43    48    53    58    63    68
[4,]    46    51    56    61    66    71    76
```

```
myTercerMatriz <- matrix(seq(1:14), 7, 2, byrow = TRUE)
```

```
myTercerMatriz
      [,1] [,2]
[1,]     1     2
[2,]     3     4
[3,]     5     6
[4,]     7     8
[5,]     9    10
[6,]    11    12
[7,]    13    14
```

```
myOtraMatriz %*% myTercerMatriz # Producto matricial
```

```
      [,1] [,2]
[1,]   742  840
[2,]  1085 1232
[3,]  1428 1624
[4,]  1771 2016
```

Puedo ampliar matrices:²⁰

```
myOtraMatrizAmpliada <- cbind(c(-1,-2,-3,-4), myOtraMatriz)
```

```
myOtraMatrizAmpliada
      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8]
[1,]    -1    11    12    13    14    15    16    17
[2,]    -2    18    19    20    21    22    23    24
[3,]    -3    25    26    27    28    29    30    31
[4,]    -4    32    33    34    35    36    37    38
```

Puedo aplicar vectorización en una matriz:

```
myOtraMatriz
```

```
myOtraMatriz + 10
      [,1] [,2] [,3] [,4] [,5] [,6] [,7]
[1,]    21    22    23    24    25    26    27
```

²⁰ Nuevamente podemos ver el uso de *cbind()*

```
[2,] 28 29 30 31 32 33 34
[3,] 35 36 37 38 39 40 41
[4,] 42 43 44 45 46 47 48
```

```
myOtraMatriz * 10
```

```
  [,1] [,2] [,3] [,4] [,5] [,6] [,7]
[1,] 110 120 130 140 150 160 170
[2,] 180 190 200 210 220 230 240
[3,] 250 260 270 280 290 300 310
[4,] 320 330 340 350 360 370 380
```

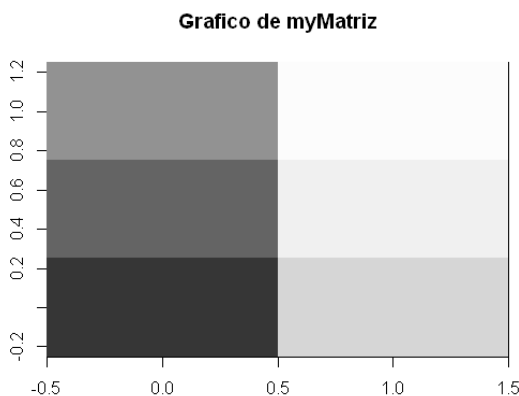
¿Qué pasa si hago un summary?:²¹

```
summary(myMatriz)
```

Co101	Co102	Co103
Min. :1.00	Min. :2.00	Min. :3.00
1st Qu.:1.75	1st Qu.:2.75	1st Qu.:3.75
Median :2.50	Median :3.50	Median :4.50
Mean :2.50	Mean :3.50	Mean :4.50
3rd Qu.:3.25	3rd Qu.:4.25	3rd Qu.:5.25
Max. :4.00	Max. :5.00	Max. :6.00

Podemos hacer cosas otras interesantes también:

```
print(image(myMatriz, main = "Grafico de myMatriz"))
```

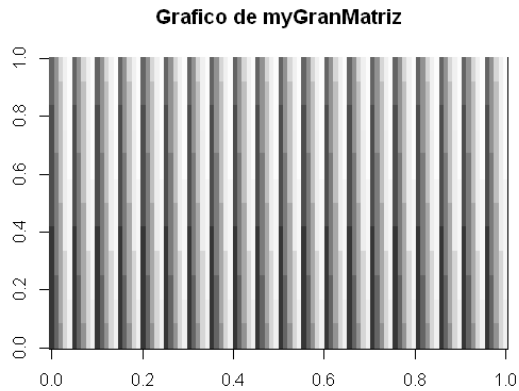


Claro, no sirve para nada lo que hicimos, pero pensemos en esto²²:

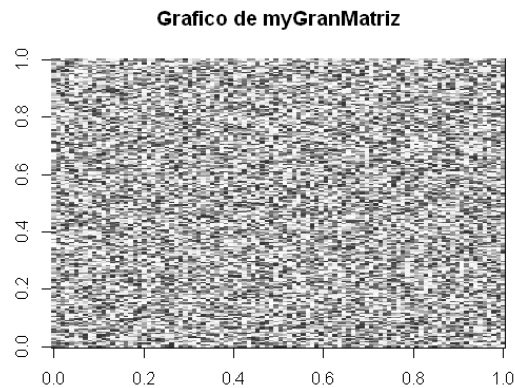
²¹ Observen como *summary()* (y muchas otras funciones de R) se adapta a la clase del objeto sobre la cual la aplico.

²² *print* es una función para imprimir en un dispositivo de salida (la pantalla por defecto) lo que le pasemos por parámetro. *image* genera una imagen representativa de un conjunto de datos. *runif* genera números aleatorios de distribución uniforme (es similar a *rnorm*, salvo que ésta aplica a distribuciones normales). En este ejemplo “graficamos” la distribución de valores numéricos en

```
myGranMatriz <- matrix(rep(seq(1:1000), times = 20),100,200,
byrow = TRUE)
print(image(myGranMatriz, main = "Grafico de myGranMatriz"))
```



```
myGranMatriz <- matrix(runif(100*200,0,50) ,100,200, byrow =
TRUE)
print(image(myGranMatriz, main = "Grafico de myGranMatriz"))
```



Podemos calcular la inversa de una matriz:

```
myMatrizCuadrada <- matrix(seq(1:4),2,2,byrow=TRUE)
```

la matriz. A priori, lo podríamos utilizar para diferenciar visualmente ruido aleatorio de alguna otra estructura subyacente.

```
myMatrizCuadrada
```

```
      [,1] [,2]  
[1,]    1    2  
[2,]    3    4
```

```
solve(myMatrizCuadrada)
```

```
      [,1] [,2]  
[1,] -2.0  1.0  
[2,]  1.5 -0.5
```

La función *solve()* tiene otro uso interesante. Se puede utilizar para resolver sistemas de ecuaciones²³:

```
matrizCoeficientes <- matrix(c(1,2,3,-1,-4,-10, 0,3,1), 3,  
3, byrow=TRUE)
```

```
vectorTermIndep <- c(0, -1, -25)
```

```
vectorSolucion <- (solve(matrizCoeficientes,  
vectorTermIndep))
```

```
vectorSolucion
```

```
[1] 10.157895 -9.263158  2.789474
```

```
matrizCoeficientes %*% vectorSolucion == vectorTermIndep
```

```
      [,1]  
[1,] FALSE  
[2,] FALSE  
[3,]  TRUE
```

¿Qué pasó?, ¿el producto no debería haber sido igual al vector de los términos independientes?:

```
matrizCoeficientes %*% vectorSolucion - vectorTermIndep
```

```
      [,1]  
[1,] -4.440892e-16  
[2,]  2.664535e-15  
[3,]  0.000000e+00
```

Lo que sucede es que, como cualquier otro método numérico, los resultados son aproximaciones. Debemos tener esto en cuenta al utilizar automáticamente los resultados de cualquier cálculo.

Con las matrices, podemos usar la función *apply()* para calcular funciones por columna o fila:

²³ En su forma de uso más general, *solve* resuelve ecuaciones de la forma $a \% \% x = b$, con x y b pudiendo ser vectores o matrices.

```
apply(z, 2, mean) # Promedio por columnas
[1] 0.5 1.0
```

```
apply(z, 1, mean) # Promedio por filas
[1] 1.0 0.5
```

Tenemos a nuestra disposición también los productos externos. El resultado de este es una matriz cuyo vector de dimensión es igual a la concatenación de los vectores de dimensiones de los dos vectores a multiplicar, y su contenido consiste en todas las multiplicaciones posibles entre los elementos de los dos elementos:

```
z %o% y
```

```
, , 1, 1
```

```
      [,1] [,2]
[1,]    1    1
[2,]    0    1
```

```
, , 2, 1
```

```
      [,1] [,2]
[1,]    1    1
[2,]    0    1
```

```
, , 1, 2
```

```
      [,1] [,2]
[1,]    0    0
[2,]    0    0
```

```
, , 2, 2
```

```
      [,1] [,2]
[1,]    1    1
[2,]    0    1
```

```
, , 1, 3
```

```
      [,1] [,2]
[1,]    1    1
[2,]    0    1
```

```
, , 2, 3
```

```
      [,1] [,2]
[1,]    1    1
[2,]    0    1
```

Para hacer lo mismo podemos utilizar la función *outer()*:

```
outer(z,y, "*")
```

E incluso podemos reemplazar la operación "*" por cualquier operación:

```
outer(z,y, "+")
```

```
outer(z,y, "-")
```


2.4. Arrays

Los arrays son generalizaciones de los vectores y las matrices. Un vector se puede utilizar como un array multidimensional mediante la función *dim()*. Un array se compone de un vector de datos y un vector de dimensión, este último es el que se declara con *dim()*:

```
z<-numeric(3*5*10) # Creo un vector de 150 elementos,  
# todos con valor cero
```

```
dim(z)<-c(3,5,10)
```

```
z
```

```
, , 1
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	0	0	0	0	0
[2,]	0	0	0	0	0
[3,]	0	0	0	0	0

```
, , 2
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	0	0	0	0	0
[2,]	0	0	0	0	0
[3,]	0	0	0	0	0

```
, , 3
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	0	0	0	0	0
[2,]	0	0	0	0	0
[3,]	0	0	0	0	0

```
, , 4
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	0	0	0	0	0
[2,]	0	0	0	0	0
[3,]	0	0	0	0	0

```
, , 5
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	0	0	0	0	0
[2,]	0	0	0	0	0
[3,]	0	0	0	0	0

```
, , 6
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	0	0	0	0	0
[2,]	0	0	0	0	0
[3,]	0	0	0	0	0

```
, , 7
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	0	0	0	0	0
[2,]	0	0	0	0	0
[3,]	0	0	0	0	0

```
, , 8
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	0	0	0	0	0

```
[2,] 0 0 0 0 0
[3,] 0 0 0 0 0
```

```
, , 9
```

```
      [,1] [,2] [,3] [,4] [,5]
[1,] 0    0    0    0    0
[2,] 0    0    0    0    0
[3,] 0    0    0    0    0
```

```
, , 10
```

```
      [,1] [,2] [,3] [,4] [,5]
[1,] 0    0    0    0    0
[2,] 0    0    0    0    0
[3,] 0    0    0    0    0
```

Con esto digo que “z” es un vector con tres dimensiones, con 3 componentes en la primera, 5 en la segunda y 10 en la tercera. Observen que fue necesario declarar la longitud, o la cantidad de componentes, del vector ($150 = 3 \times 5 \times 10$) con el comando `dim()`, y luego pasarle el vector de cómo se deben distribuir las componentes entre las dimensiones (`c(3,5,10)`):

```
z[1,4,7]<-25
```

```
z[1,4,7]
```

```
[1] 25
```

```
z[1,4,]
```

```
[1] 0 0 0 0 0 0 25 0 0 0
```

```
z[1,,]
```

```
      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
[1,] 0    0    0    0    0    0    0    0    0    0
[2,] 0    0    0    0    0    0    0    0    0    0
[3,] 0    0    0    0    0    0    0    0    0    0
[4,] 0    0    0    0    0    0    25   0    0    0
[5,] 0    0    0    0    0    0    0    0    0    0
```

También podría haberlo declarado con la función `array()`, donde el primer parámetro es el vector de datos y el segundo el de dimensión:

```
x<-c(1:10)
```

```
y<-c(5,2)
```

```
z<-array(x,y)
```

```
z
```

```
      [,1] [,2]
[1,] 1    6
[2,] 2    7
[3,] 3    8
[4,] 4    9
[5,] 5   10
```

Los arrays nos permiten trabajar con subarrays:

```
x<-array(c(1,5,4,2,2,1), c(3,2))
```

```

x
  [,1] [,2]
[1,]  1  2
[2,]  5  2
[3,]  4  1

z[x] # Pido que devuelva los elementos (1,2), (5,2) y (4,1)
[1] 6 10 4

z[x]<-0
z
  [,1] [,2]
[1,]  1  0
[2,]  2  7
[3,]  3  8
[4,]  0  9
[5,]  5  0

```

2.5. Listas

Las "list" son colecciones ordenadas de otros objetos (incluso de otras listas). Estos objetos pueden ser de cualquier tipo y disimiles entre sí:

```

myList <- list(nombre="Pedro", valorLogico=TRUE,
               unNumero=3, unVector=c(4,7,9),
               unaMatriz=matrix(c(1,0,1,1), nrow=2))

```

```

myList
$nombre
[1] "Pedro"

$valLogico
[1] TRUE

$unNumero
[1] 3

$unVector
[1] 4 7 9

$unaMatriz
  [,1] [,2]
[1,]  1  1
[2,]  0  1

```

¿No les parece conocido esto?. Es que los data.frame son un caso especial de listas en el cual cada elemento es un vector de la misma longitud.

Volviendo a las listas, se puede acceder a cada elemento mediante su nombre:

```

myList$unaMatriz
  [,1] [,2]
[1,]  1  1
[2,]  0  1

```

O bien mediante dobles corchetes "[[]]" referenciando al índice del elemento:

```
myList[[5]]  
      [,1] [,2]  
[1,]    1    1  
[2,]    0    1
```

Con corchetes simples accedo la sublista:

```
myList[5]  
$unaMatriz  
      [,1] [,2]  
[1,]    1    1  
[2,]    0    1
```

Observen la diferencia, con corchetes simples devolvemos un objeto tipo lista (incluye el nombre de la componente por ejemplo), pero con dobles obtenemos el objeto almacenado:

```
class(myList[1])  
[1] "list"  
  
class(myList[[1]])  
[1] "character"
```

Puedo crear la lista sin nombre y asignárselos después:

```
myList<- list("Juan", "25", FALSE)  
myList  
[[1]]  
[1] "Juan"  
  
[[2]]  
[1] "25"  
  
[[3]]  
[1] FALSE  
  
names(myList)<- c("nombre", "numero", "booleano")  
myList  
$nombre  
[1] "Juan"  
  
$numero  
[1] "25"  
  
$booleano  
[1] FALSE
```

length() me permite conocer el tamaño de la lista:

```
length(myList)  
[1] 3
```

Puedo concatenar listas también:

```
myList2 <- list(matrix(c(1,0,1,1), nrow=2),  
                 matrix(c(1,1,1,0), nrow=2))  
myList3 <- c(myList, myList2)  
myList3  
$nombre  
[1] "Juan"  
  
$numero  
[1] "25"  
  
$booleano  
[1] FALSE  
  
[[4]]  
      [,1] [,2]  
[1,]    1    1  
[2,]    0    1  
  
[[5]]  
      [,1] [,2]  
[1,]    1    1  
[2,]    1    0
```

Un poco más de R

3.1. Utilizando funciones en R

R tiene muchas funciones predefinidas, pero muchas veces es útil definir nuestras propias funciones. Definir funciones en R es muy fácil:

```
mySuma <- function(x,y) {  
  x+y  
}
```

Acabamos de definir una función que se llama *mySuma* y que acepta dos parámetros, “x” e “y”. El resultado que devuelve es la suma de los dos parámetros pasados al invocarla:

```
mySuma(10,25)  
[1] 35
```

En R no hace falta el uso de un comando del tipo “return” para indicar que valor debe devolver la función. En R, cualquier función devuelve el último objeto que se llame o invoque durante su ejecución. En el caso de la función *mySuma*, el resultado de la suma.

Si quiero ver cómo está formada mi función, la llamo sin parámetros:

```
mySuma  
function(x,y) {  
  x+y  
}
```

Las funciones en R son muy interesantes. A efectos prácticos, son un objeto más del sistema, así que las puedo asignar a variables:

```
algunaFuncion <- mySuma  
  
algunaFuncion(2,3)  
[1] 5  
  
algunaFuncion  
function(x,y) {  
  x+y  
}
```

Entonces, puedo hacer una función que tome como parámetros otras funciones:²⁴

```
myResta <- function(x,y) {  
  x-y  
}  
  
myOperacion <- function(x,y,cualquierOtraFuncion){  
  cualquierOtraFuncion (x,y)  
}  
  
myOperacion(2,3,mySuma)  
myOperacion(2,3,myResta)
```

Fijémonos que la función *myOperacion* toma el tercer parámetro (la función a aplicar) y lo ejecuta con el primer y segundo parámetro. Es interesante que, gracias a la vectorización, aunque no lo pensamos, funcionan igual cosas como estas:

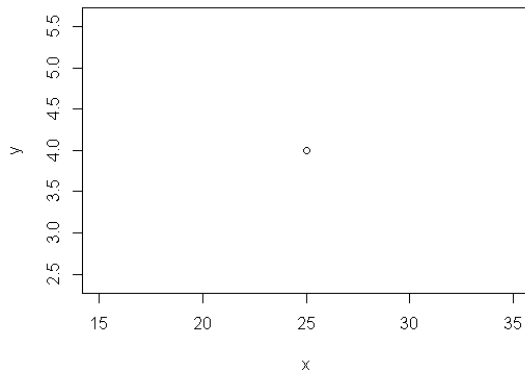
```
secuencia <- seq(1:20)  
mySuma(secuencia, 100)  
[1] 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116  
[17] 117 118 119 120
```

!!!La función aplica vectorización y reciclado en forma automática!!!!, sumó 100 a cada elemento del vector “secuencia”.

Hasta ahora nuestra función es "muy matemática" pero el concepto de función en R, como en casi cualquier otro lenguaje de programación, está pensado desde el punto de vista algorítmico.

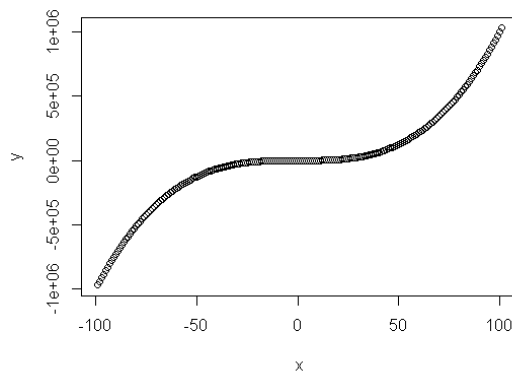
```
myGraficador <- function(x,y){  
  plot(x,y)  
}  
  
myGraficador(25,4)
```

²⁴ Básicamente, así funcionan *sapply()* y *tapply()*.



Que es una función algorítmica. Mediante vectorización:

```
myGraficador(seq(0:200)-100, (seq(0:200)-100)^3)
```

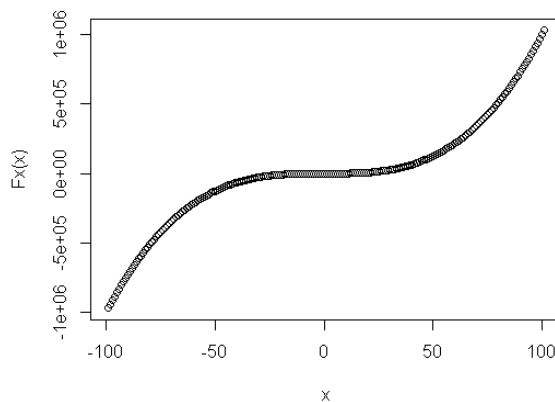


Si lo pensamos, *myGraficador()* es una función horrible. Le tenemos que pasar los valores de las abscisas y de las ordenadas para graficar, pero no le podemos pasar un conjunto de valores y una función para aplicarles. Arreglémoslo un poco:

```
myGraficadorDeFunciones <- function(x, Fx){
  plot(x, Fx(x))
}
```


Listo, tenemos un graficador que, pasándole el dominio ("x") y una función a aplicar ("Fx") genera la gráfica correspondiente en 2D. Definamos ahora una función cúbica:

```
myCubica <- function(x){
  x^3
}
myGraficadorDeFunciones(seq(0:200)-100, myCubica)
```



Sigue siendo muy fea, ya que a myCubica() no le puedo pasar parámetros. Redefinámosla:

```
myCubica <- function(x, coefa, coefb, coefc, coefd){
  coefa*(x^3) + coefb*(x^2) + coefc*x + coefd
}
myGraficadorDeFunciones(seq(0:200)-100, myCubica)
Error en coefa * (x^3) : coefa' esta perdido
```

¿Qué pasó?. Nos tiró un error porque le agregamos nuevos parámetros a myCubica() y no los estamos pasando desde myGraficadorDeFunciones(). Redefinamos esta última:

```
myGraficadorDeFunciones <- function(x, Fx, ...){
  plot(x, Fx(x, ...))
}
```

Con "." le decimos a R que myGraficadorDeFunciones puede tomar un número indeterminado y no obligatorio de parámetros adicionales a los que si declaramos obligatoriamente (x y Fx), y con el mismo operador podemos hacer uso de ellos dentro de la función. Veamos su ejecución:

```
myGraficadorDeFunciones(seq(0:200)-100, myCubica, -3,20,0,-4)
```

Las funciones en R permiten el pasaje de parámetros nombrados. Por ejemplo, definamos esta función:

```
potencia <- function(base, exponente){  
  base^exponente  
}
```

Dada esta función, la puedo llamar de la manera convencional, pasando los parámetros en orden:

```
potencia(10,2)  
[1] 100
```

Pero no tiene por qué ser así, puedo pasarle los parámetros desordenados siempre y cuando los pase con nombre:²⁵

```
potencia(exponente=2, base=10)  
[1] 100
```

También puedo definir una función con valores de parámetros por defecto, que hacen opcional el ingreso de los mismos:

```
potencia <- function(base, exponente=2){  
  base^exponente  
}
```

Si la llamo sin declararle el exponente:

```
potencia(base=10)  
[1] 100
```

²⁵ Observemos el detalle que para pasar valores a parámetros si uso el operador "=", en vez del operador "<="

Y, ya que base es el primer parámetro, puedo omitir su nombre:²⁶

```
potencia(10)
[1] 100
```

3.2. Guardando nuestras funciones

Obviamente, si tenemos que escribir las funciones cada vez que las queremos usar, su uso sería bastante limitado. R nos permite definir scripts, archivos de texto plano con extensión “.R” que contienen código R. Así que para grabar nuestras funciones y tenerlas disponibles cuando queramos, basta crear un archivo de texto, escribir ahí las definiciones de las funciones, y guardarlo con extensión “.R”. Por ejemplo, creemos el archivo “funcionesPrueba.R” con cualquier editor de texto (o desde el panel “Source” del RStudio)²⁷ y escribamos lo siguiente:

```
potencia <- function(base, exponente=2){
  base^exponente
}

raiz <- function(base, indice=2){
  base^(1/indice)
}
```

Guardemos el archivo en la carpeta “sources” de nuestro directorio de trabajo, y ejecutemos el siguiente código:

```
rm(list=ls())
character(0)
source("../sources\\funcionesPrueba.R")
ls()
```

²⁶ Igual tengamos cuidado con esto. En general, conviene declarar, cuando se crea una función, todos los parámetros obligatorios primero y después los parámetros con valores por defecto. Pero ante la duda, al momento de llamar a la función, nombremos el parámetro.

²⁷ En estas cosas vemos como conviene utilizar un IDE integrado como RStudio en vez de solamente la consola básica de R.

```
[1] "potencia" "raiz"
```

Llamamos a `rm()` para asegurar que no tenemos ninguna función con este nombre creada, con `source()` leemos el archivo recién creado, y con `ls()` mostramos los objetos disponibles en la sesión (apareciendo las nuevas funciones). Probemos usar alguna:

```
raiz(10,2)
[1] 3.162278
```

Excelente, funcionan correctamente.

3.3. Armando scripts

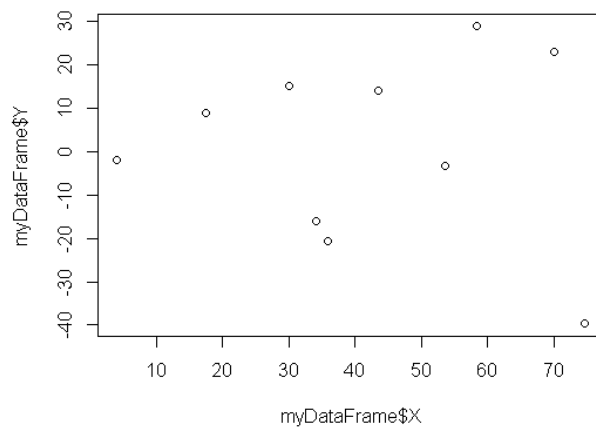
El archivo que creamos solo tenía cargado definiciones de funciones, pero también puedo alojar allí código que quiero que se ejecute al leer el archivo de script. Creemos en nuestro directorio `script` el archivo “`scriptPrueba.R`” y carguémosle el siguiente contenido:

```
# Script de prueba en R
# Creamos un data.frame
myDataFrame <- data.frame(runif(10,0,100), runif(10,-50,50))
names(myDataFrame) <- c("X","Y")
plot(myDataFrame$X, myDataFrame$Y)
View(myDataFrame)
```

Ahora ejecutemos el comando `source()`.²⁸

```
source("scriptPrueba.R")
```

²⁸ Como mi directorio de trabajo es “`script`”, no es necesario pasarle ninguna ruta, basta con el nombre del archivo.



	X	Y
1	74.701114	-39.728437
2	35.819991	-20.666453
3	17.464645	8.962565
4	34.191171	-16.073070
5	53.537047	-3.202527
6	29.998742	14.988526
7	58.320702	28.838195
8	43.490644	13.963918
9	4.039695	-1.901415
10	69.998986	23.034759

Gráficos con R

4.1. Trabajando con el paquete *graphics*

Vamos a presentar una pequeña introducción al uso de gráficos con R.

Para ello, vamos a necesitar el archivo de datos

“datosPoblacionMaraton.csv”²⁹

Arranquemos nuestro directorio de trabajo:

```
setwd("D://Proyectos//IOR//scripts")
```

Y limpiemos la memoria de trabajo:

```
rm(list=ls())
```

Comencemos cargando el archivo "datosPoblacionMaraton.csv" en la variable “dPM”:

```
dPM <- read.csv("../data//datosPoblacionMaraton.csv")
```

Es una buena práctica revisar el data frame cargado antes de ponernos a trabajar. Veamos su estructura:

```
str(dPM)
```

```
'data.frame':      6000 obs. of  7 variables:
 $ X                : int  1 2 3 4 5 6 7 8 9 10 ...
 $ sexo             : Factor w/ 2 levels "F","M":  1 2 2
 1 1 2 2 2 1 2 ...
 $ altura           : num  1.69 1.71 1.71 1.71 1.89 ...
 $ peso            : num  77.9 79.5 79.6 79.8 88.1 ...
 $ edad            : num  45.2 47.1 41.6 50.6 43.2 ...
 $ frecuenciaSemanalEntrenamiento: int  3 4 0 4 3 2 2 3 3 4 ...
 $ tiempo21KMMin   : num  98.1 79.8 160 82.5 122.2 ...
```

Veamos un poco algunos registros:

```
head(dPM)
```

```
  X sexo  altura  peso  edad frecuenciaSemanalEntrenamiento
```

²⁹ Se puede descargar desde

“<http://www.modelizandosistemas.com.ar/p/optimizacion-con-r.html>”. Una vez descargado, hay que copiarlo en la carpeta “data” de nuestro proyecto.

```

1 1 F 1.686767 77.86853 45.23844 3
2 2 M 1.707745 79.50636 47.10703 4
3 3 M 1.708544 79.56911 41.55911 0
4 4 F 1.710859 79.75118 50.57101 4
5 5 F 1.891673 88.08078 43.18842 3
6 6 M 1.843027 83.90220 39.16556 2
tiempo21KMMin
1 98.14615
2 79.82995
3 159.99911
4 82.46038
5 122.19544
6 136.86943

```

Podemos hacer una visualización completa también:

View(dPM)

	X	sexo	altura	peso	edad	frecuenciaSemanalEntrenamiento	tiempo21KMMin
1	1	F	1.686767	77.86853	45.23844	3	98.14615
2	2	M	1.707745	79.50636	47.10703	4	79.82995
3	3	M	1.708544	79.56911	41.55911	0	159.99911
4	4	F	1.710859	79.75118	50.57101	4	82.46038
5	5	F	1.891673	88.08078	43.18842	3	122.19544
6	6	M	1.843027	83.90220	39.16556	2	136.86943
7	7	M	1.690187	78.13413	32.36071	2	110.35485
8	8	M	1.752911	76.44902	46.47301	3	114.72397
9	9	F	1.871720	86.35375	47.52661	3	123.78147
10	10	M	1.875104	86.64532	42.24352	4	99.32822
11	11	M	1.681953	70.84323	34.35950	2	122.53565
12	12	M	1.970735	95.10433	27.52454	4	96.13051

Displayed 1000 rows of 6000 (5000 omitted)

OK, ya vimos cómo está formado el conjunto de datos, familiaricémonos con su contenido:

summary(dPM)

```

      X      sexo      altura      peso      edad
Min.   : 1 F:2995 Min.   :1.482 Min.   : 56.31 Min.   :12.15
1st Qu.:1501 M:3005 1st Qu.:1.668 1st Qu.: 74.82 1st Qu.:38.18
Median :3000      Median :1.719 Median : 78.30 Median :42.49
Mean   :3000      Mean   :1.741 Mean   : 79.02 Mean   :42.55
3rd Qu.:4500      3rd Qu.:1.805 3rd Qu.: 82.56 3rd Qu.:46.99
Max.   :6000      Max.   :2.177 Max.   :114.81 Max.   :66.94

frecuenciaSemanalEntrenamiento tiempo21KMMin
Min.   :0.00 Min.   : 60.27
1st Qu.:1.00 1st Qu.:101.10
Median :2.00 Median :119.31
Mean   :2.25 Mean   :121.44
3rd Qu.:3.00 3rd Qu.:139.68
Max.   :4.00 Max.   :201.68

```

Bueno, comencemos a trabajar. El último campo del data frame es el tiempo que tardó cada maratonista en recorrer los 21KM. En general, se

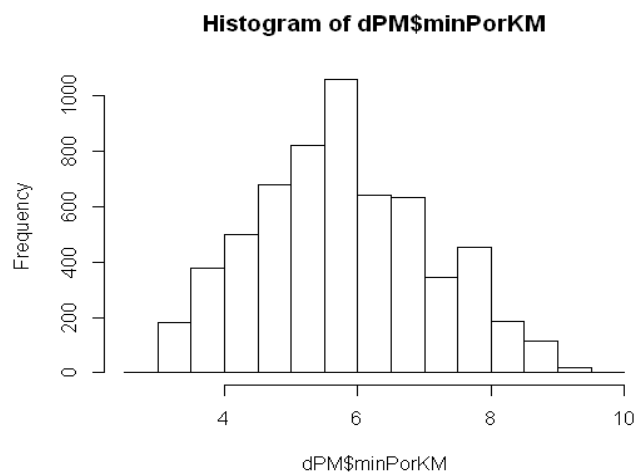
suele utilizar como medida de performance el tiempo (en minutos) que se tarda en recorrer 1KM, así que creemos ese campo:

```
dPM$minPorKM <- dPM$tiempo21KMMin/21
```

Veamos ahora como se distribuyeron las velocidades:

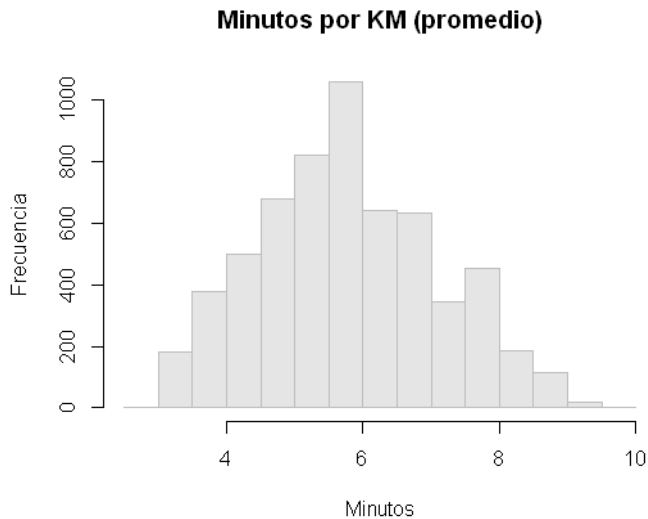
```
summary(dPM$minPorKM)  
hist(dPM$minPorKM)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
2.870	4.814	5.682	5.783	6.651	9.604



Mejoremos un poquito el gráfico

```
hist_minPorKM <- hist(dPM$minPorKM, col = "gray90",  
border="gray", main = "Minutos por KM (promedio)", xlab =  
"Minutos", ylab = "Frecuencia", axes = TRUE)
```

Fíjense que guardamos el histograma en una variable. Y es que, como decíamos en anteriores capítulos, casi todo en R es un objeto, y todos los objetos se pueden guardar en variables. Si averiguamos su clase:

```
class(hist_minPorKM)
```

```
[1] "histogram"
```

Como todo objeto, podemos analizar su estructura:

```
str(hist_minPorKM)
```

```
List of 7
 $ breaks      : num [1:16] 2.5 3 3.5 4 4.5 5 5.5 6 6.5 7 ...
 $ counts      : int [1:15] 2 182 379 498 676 819 1056 641 633 342
 ...
 $ intensities: num [1:15] 0.000667 0.060667 0.126333 0.166 0.225333
 ...
 $ density     : num [1:15] 0.000667 0.060667 0.126333 0.166 0.225333
 ...
 $ mids        : num [1:15] 2.75 3.25 3.75 4.25 4.75 5.25 5.75 6.25
 6.75 7.25 ...
 $ xname       : chr "dPM$minPorKM"
 $ equidist    : logi TRUE
 - attr(*, "class")= chr "histogram"
```

Y claro, sabiendo su estructura podemos utilizarla para otras cosas:

```
View(data.frame(hist_minPorKM$mids, hist_minPorKM$counts))
```

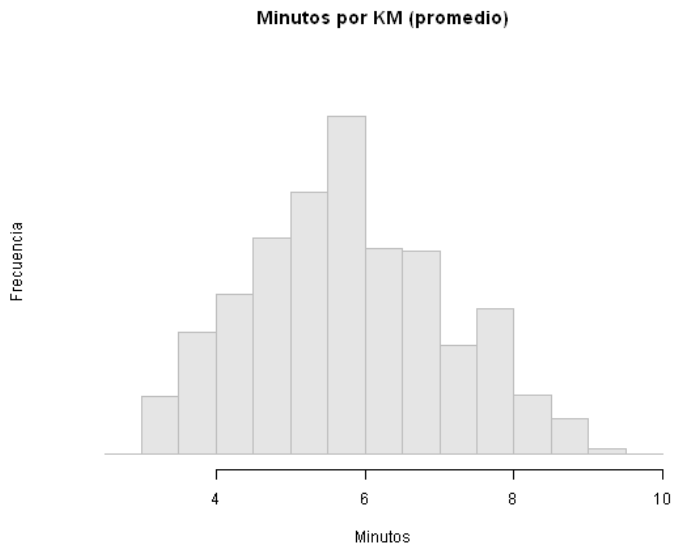
	hist_minPorKM.mids	hist_minPorKM.counts
1	2.75	2
2	3.25	182
3	3.75	379
4	4.25	498
5	4.75	676
6	5.25	819
7	5.75	1056
8	6.25	641
9	6.75	633
10	7.25	342
11	7.75	453
12	8.25	186

Pero sigamos mejorando el gráfico. Vamos a recrearlo un poco. Ojo cuando utilicemos la función *par()*, guardemos antes los valores originales y restaurémoslos después de usarlos:

```
par_cex <- par("cex") # Guardamos el valor de "cex"
par(cex=0.75) # Reducimos la proporción de la fuente
par_yaxt <- par("yaxt") # Guardamos el valor de "yaxt"
par(yaxt="n") # Quitamos el eje Y
par_bty <- par("bty") # Guardamos el valor de "bty"
par(bty = "n") # Sacamos el borde del gráfico

hist_minPorKM <- hist(dPM$minPorKM, col = "gray90",
border="gray", main = "Minutos por KM (promedio)", xlab =
"Minutos", ylab = "Frecuencia", axes = TRUE, labels = FALSE,
ylim = c(0,1200))

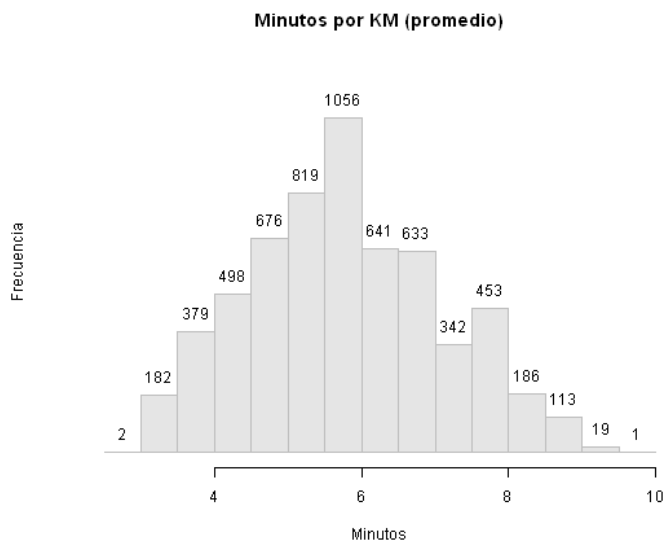
# Revertimos los cambios, devolviendo a las variables los
valores originales.
par(cex=par_cex)
par(yaxt=par_yaxt)
par(bty = par_bty)
```



par permite acceder a valores globales de configuración de gráficos. Si no reversionamos los cambios, el próximo gráfico saldrá con la misma configuración.

Agreguemos los valores de frecuencia:

```
text(hist_minPorKM$mids, hist_minPorKM$counts,
hist_minPorKM$counts, adj=c(.5,-1), cex = 0.75)
```



El primer parámetro pasado a *text()* le dice la posición en X en la que tiene que escribir, el segundo la posición en Y, el tercero el texto a escribir, el cuarto el desplazamiento respecto de los valores X e Y, y el quinto reduce el tamaño de la letra. Como quiero poner el valor que corresponde a cada barra encima de esta, $X = \text{hist_minPorKM\$mids}$ (el centro de cada barra) e $Y = \text{hist_minPorKM\$counts}$ (la altura de la barra).

Superpongámosle ahora un polígono de frecuencia. Primero, calculo el ancho de clase del histograma:

```
hist_minPorKM.anchoclase <-  
hist_minPorKM$mids[2]- hist_minPorKM$mids[1]
```

Calculo los vectores de coordenadas horizontales y verticales:

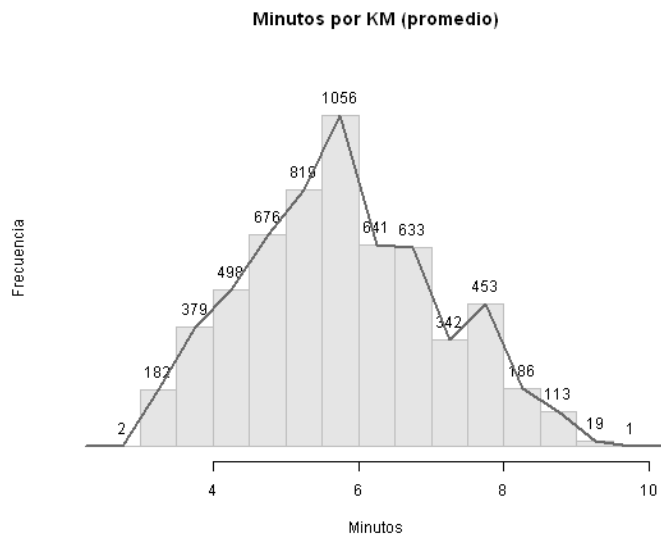
```
xx <- c( hist_minPorKM$mids[1]-hist_minPorKM.anchoclase,  
hist_minPorKM$mids,  
tail(hist_minPorKM$mids,1)+hist_minPorKM.anchoclase )  
yy <- c(0, hist_minPorKM$counts, 0)
```

El vector "xx" tiene como primer y último elementos valores, por afuera del rango de graficación del histograma (al primer elemento le resto un ancho de clase, y al último, obtenido con *tail()*, le sumo un ancho de clase), el resto de los valores son los centros de cada clase del histograma. El vector "yy" inicia y termina con 0, y el resto de los valores son la frecuencia absoluta de cada clase del histograma.

Agrego el polígono al histograma:

```
lines(xx, yy, lwd=2, col = "royalblue")
```

Donde "lwd" es el ancho de la línea.



Volvamos ahora a nuestro data frame. ¿Cuántos valores cayeron por debajo de la media?:

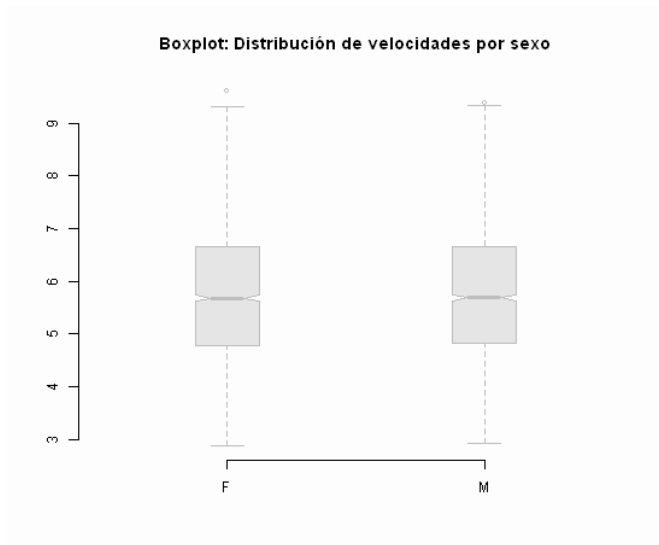
```
length(dPM$minPorKM[dPM$minPorKM < mean(dPM$minPorKM)])
```

Tendrá alguna relación el sexo con la velocidad promedio:

```
par_cex <- par("cex")
par(cex=0.75)
par_bg <- par("bg")
par(bg = "lightyellow") # Color del fondo
par_yaxt <- par("yaxt")
par(yaxt="s") #Agrego el eje y
par_bty <- par("bty")
par(bty = "n")

box_minVsSexo <- boxplot(minPorKM ~ sexo, data = dPM, col =
"gray90", border = "grey", notch=TRUE, boxwex=0.25 )
title("Boxplot: Distribución de velocidades por sexo")

par(cex=par_cex)
par(bg = par_bg)
par(yaxt = par_yaxt)
par(bty = par_bty)
```



Observen que restauré los valores de *par*. Como con cualquier otro objeto, puedo consultar la variable en la que almacené mi boxplot:

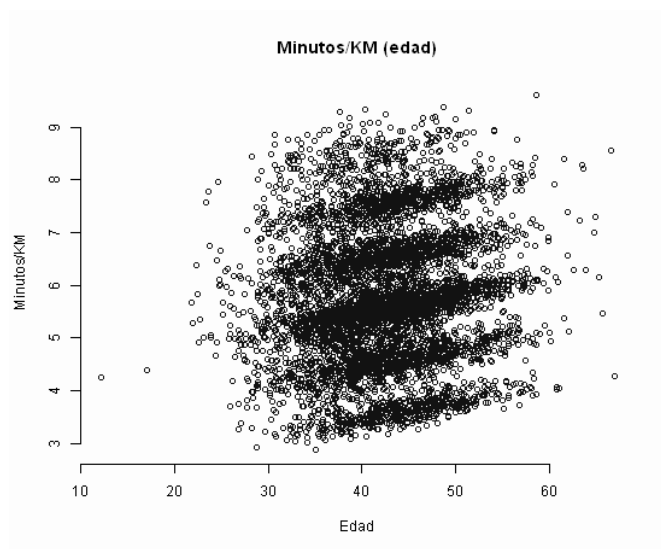
```
str(box_minVsSexo)
```

Posiblemente la velocidad esté relacionada con la edad:

```
par_cex <- par("cex")
par(cex=0.75)
par_bg <- par("bg")
par(bg = "lightyellow")
par_bty <- par("bty")
par(bty = "n")

plot_minVsEdad <- plot(dPM$edad, dPM$minPorKM, main =
  "Minutos/KM (edad)", xlab="Edad", ylab="Minutos/KM",
  col="blue")

par(cex=par_cex)
par(bg = par_bg)
par(bty = par_bty)
```

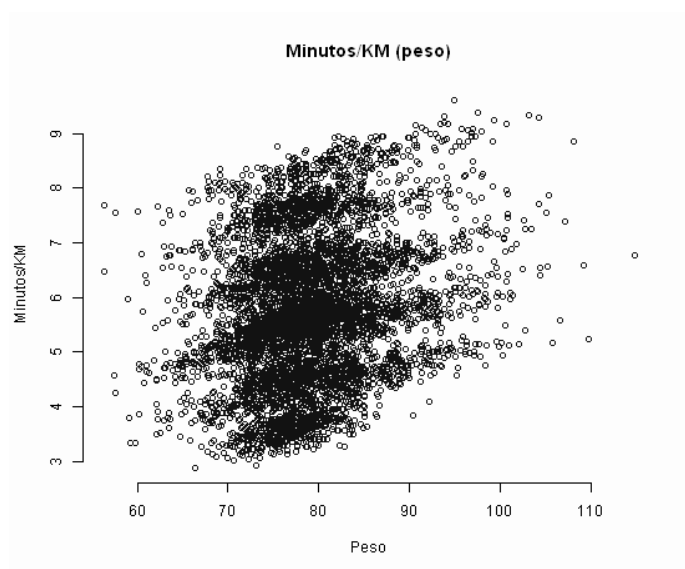


¿Y con el peso?:

```
par_cex <- par("cex")
par(cex=0.75)
par_bg <- par("bg")
par(bg = "lightyellow")
par_bty <- par("bty")
par(bty = "n")

plot_minVsPeso <- plot(dPM$peso, dPM$minPorKM, main =
"Minutos/KM (peso)", xlab="Peso", ylab="Minutos/KM",
col="blue")

par(cex=par_cex)
par(bg = par_bg)
par(bty = par_bty)
```

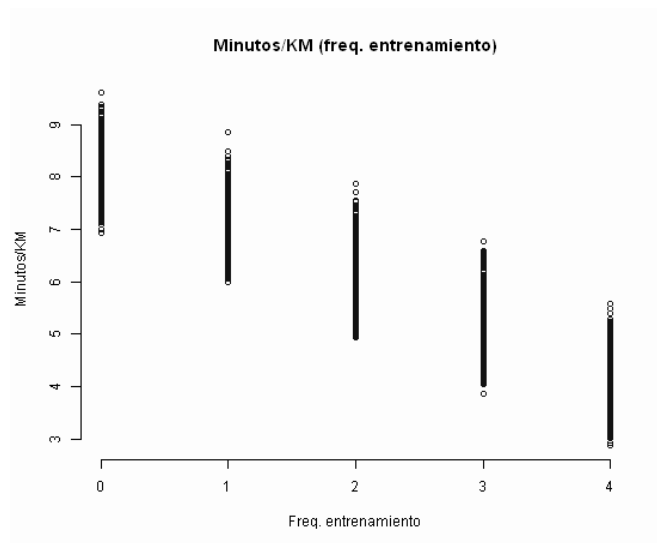


¿Y cómo varía con la frecuencia de entrenamiento?:

```
par_cex <- par("cex")
par(cex=0.75)
par_bg <- par("bg")
par(bg = "lightyellow")
par_bty <- par("bty")
par(bty = "n")

plot_minVsFreq <- plot(dPM$frecuenciaSemanalEntrenamiento,
dPM$minPorKM, main = "Minutos/KM (freq. entrenamiento)",
xlab="Freq. entrenamiento", ylab="Minutos/KM", col="blue")

par(cex=par_cex)
par(bg = par_bg)
par(bty = par_bty)
```

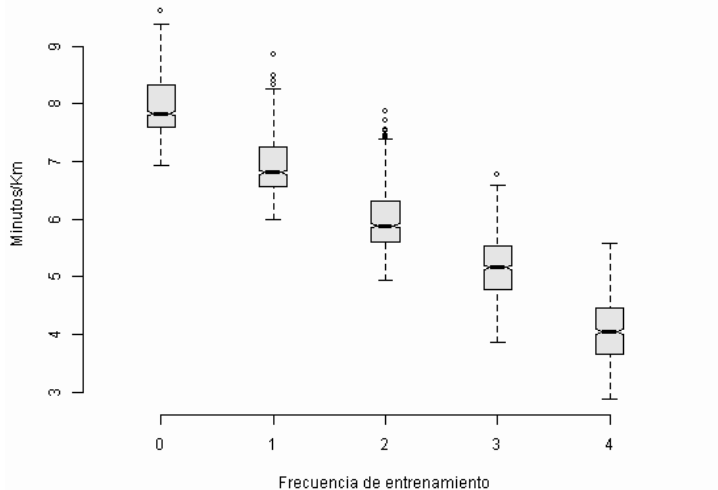
Un gráfico medio feo, mejor un boxplot:

```
par_cex <- par("cex")
par(cex=0.75)
par_bg <- par("bg")
par(bg = "lightyellow")
par_yaxt <- par("yaxt")
par(yaxt="s")
par_xaxt <- par("xaxt")
par(xaxt="s")
par_bty <- par("bty")
par(bty = "n")

box_minVsFrecuencia <- boxplot(minPorKM ~
frecuenciaSemanalEntrenamiento, data = dPM, col = "gray90",
notch=TRUE, boxwex=0.25 )
title(main="Boxplot: Distribución de velocidades frecuencia
de entrenamiento")
title(xlab="Frecuencia de entrenamiento")
title(ylab="Minutos/Km")

par(cex=par_cex)
par(bg = par_bg)
par(yaxt = par_yaxt)
par(xaxt = par_xaxt)
par(bty = par_bty)
```

Boxplot: Distribución de velocidades frecuencia de entrenamiento



Aparentemente hay alguna vinculación entre la velocidad promedio, la frecuencia de entrenamiento y las características fisiológicas. Armemos un modelo de regresión con la función *lm()*:

```
lm.vel <- lm(dPM$minPorKM ~ dPM$peso + dPM$edad +
dPM$frecuenciaSemanalEntrenamiento)
summary(lm.vel)
```

Call:

```
lm(formula = dPM$minPorKM ~ dPM$peso + dPM$edad +
dPM$frecuenciaSemanalEntrenamiento)
```

Residuals:

Min	1Q	Median	3Q	Max
-0.86541	-0.32015	-0.02829	0.32447	0.88141

Coefficients:

	Estimate	Std. Error	t value
Pr(> t)			
(Intercept)	3.2833367	0.0663475	49.49
<2e-16 ***			
dPM\$peso	0.0521247	0.0006957	74.92
<2e-16 ***			
dPM\$edad	0.0122465	0.0007130	17.18
<2e-16 ***			
dPM\$frecuenciaSemanalEntrenamiento	-0.9511652	0.0036234	-262.50
<2e-16 ***			

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.3587 on 5996 degrees of freedom
Multiple R-squared: 0.9255, Adjusted R-squared: 0.9254
F-statistic: 2.482e+04 on 3 and 5996 DF, p-value: < 2.2e-16

Así como los gráficos son objetos, los modelos de ajuste de datos son también objetos:

```
class(lm.vel)
```

Como cualquier objeto, puedo acceder a su estructura:

```
str(lm.vel)
View(lm.vel$coefficients)
```

	row.names	x
1	(Intercept)	3.28333667
2	dPM\$peso	0.05212468
3	dPM\$edad	0.01224653
4	dPM\$frecuenciaSemanalEntrenamiento	-0.95116523

```
class(lm.vel$coefficients)
```

```
[1] "numeric"
```

Vemos que “coefficients” es un vector en el cual la primera componente es el termino independiente, y el resto los coeficientes estimados para la función de velocidad. Sabiendo esto, puedo usar el modelo para predecir valores. Supongamos que quiero saber la velocidad promedio que alcanzará una persona de 75 kg de peso, 25 años de edad y una frecuencia de entrenamiento de 4 días semanales:

```
lm.vel$coefficients[1] + lm.vel$coefficients[2]*75 +  
lm.vel$coefficients[3]*25 + lm.vel$coefficients[4]*4
```

Mejor aún, puedo hacerlo así:

```
prediccion_vel <- sum(lm.vel$coefficients * c(1,75,25,4))  
prediccion_vel
```

El “1” en la primer componente del vector `c(1,75,25,4)` sirve para mantener el termino independiente del modelo en un valor fijo (ya que lo multiplico por la constante).

Claro, tiene mucha dispersión mi población, quizás pueda armar un intervalo de predicción:

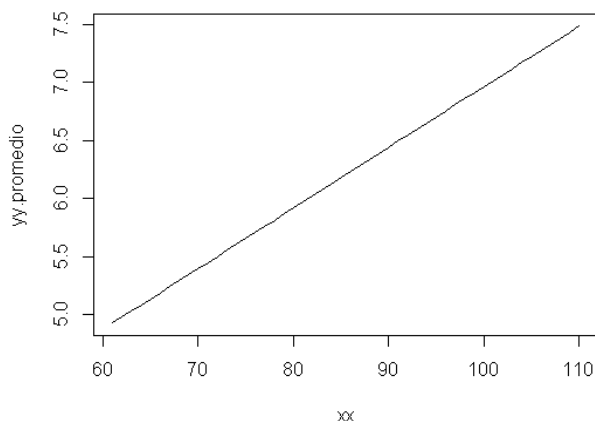
```
residuos.sd_vel <- sd(lm.vel$residuals)  
prediccion_vel_interval <- c(prediccion_vel -  
2*residuos.sd_vel, prediccion_vel + 2*residuos.sd_vel)  
  
prediccion_vel_interval
```

```
[1] 2.977059 4.411321
```

Supongamos que queremos graficar la influencia del peso dada una frecuencia de entrenamiento fija (2) y una edad también fija (30), para el rango de peso comprendido entre 61 kg y 110 kg. Creo una variable denominada “xx” para almacenar los pesos, y una familia de variables “yy” para almacenar la velocidad:

```
edad <- 30
frec <- 2
xx <- seq(1:50)+60
yy.promedio <- lm.vel$coefficients[1] +
lm.vel$coefficients[2]*xx + lm.vel$coefficients[3]*edad +
lm.vel$coefficients[4]*frec
yy.min <- yy.promedio - 2*residuos.sd_vel
yy.max <- yy.promedio + 2*residuos.sd_vel

plot(xx, yy.promedio, type = "l")
```



Donde tenemos un gráfico para buscar la velocidad promedio que alcanzará un maratonista respecto de su peso. Obviamente, en un gráfico muy feo, queda para el lector mejorarlo un poco.

4.2. Trabajando con layouts

Vamos a ver ahora como graficar más de un gráfico en, valga la redundancia, el mismo gráfico. Continuando trabajando con los datos almacenados en nuestra variable “dPM”, vamos a graficar, en la misma